

Applicability of Lehman's Laws of Software Evolution

Analysis Based on Linux Kernel Data (1994–2008)

Submitted by :

Pronob Karmoker

BSSE1431

Introduction

The figures provided track the Linux kernel's evolution across versions 1.0 (1994) to 2.6.x (2008) - a classic **E-type (Evolving)** system that operates in a real-world environment. The metrics available are:

- Number of System Calls
- Lines of Code (LOC) - All Directories
- Number of Functions - All Directories
- McCabe Cyclomatic Complexity - All Directories
- Oman's Maintainability Index - All Directories
- Releases per Month
- Kernel Release Dates (Minor Release Sequence Number)
- Incremental Change in Number of Files

Each of Lehman's eight laws is evaluated against these figures below.

Law I - Continuing Change

"An E-type system must be continually adapted, else it becomes progressively less satisfactory in use."

Verdict: Strongly Applicable

The Number of System Calls grew continuously from ~130 in v1.0 (1994) to over 350 by v2.6.x (2008). LOC expanded from near zero to approximately 5 million lines across the same period. The Releases per Month chart confirms persistent release activity from 1994 through 2003, with multiple releases each month, reflecting constant adaptation to new hardware platforms, security vulnerabilities, driver requirements, and user demands. Without this continual change, the Linux kernel would have become obsolete in a rapidly evolving computing environment - exactly what this law predicts.

Law II - Increasing Complexity

"As an E-type system evolves, its complexity increases unless work is done to maintain or reduce it."

Verdict: Strongly Applicable

The McCabe Cyclomatic Complexity chart shows a steep, sustained increase across all kernel branches - rising from approximately 19,000 in 1994 to over 700,000 by 2008. The Number of Functions chart mirrors this, growing from near zero to over 180,000 functions. Both metrics confirm that structural complexity grew with every release cycle. Although the Oman's Maintainability Index remained relatively stable (between ~ 108 and ~ 115), suggesting that some deliberate effort was made to manage complexity, the raw complexity figures make the upward trend undeniable. This is a textbook confirmation of Law II.

Law III - Self-Regulation

"E-type system evolution is self-regulating, with distribution of product and process measures close to normal."

Verdict: Partially Applicable

This law is harder to confirm directly from the provided charts since it ideally requires statistical distribution analysis. However, indirect evidence exists. The Incremental Change in Number of Files (2004-2008) oscillates around a stable average of approximately 280 files per release, with values fluctuating between ~ 0 and ~ 630 but consistently returning to the mean rather than diverging indefinitely. This oscillatory behaviour around a central value is consistent with a self-regulating system. The relatively stable Maintainability Index across branches further hints at regulated quality management. Full confirmation is limited because the figures do not show frequency distributions of all measures, so this law is only partially supported.

Law IV - Conservation of Organisational Stability (Invariant Work Rate)

"The average effective global activity rate in an evolving E-type system is invariant over the lifetime of the product."

Verdict: Partially Applicable

The Releases per Month chart (1994-2003) shows considerable variation - peaks of 12-15 releases per month during 1995-1996, dropping to 3-5 in quieter periods. The Incremental Change in Number of Files similarly shows high variance between

release cycles. However, across the entire lifespan, there is a rough average activity level that the project oscillates around rather than diverging explosively or collapsing entirely. For an open-source project where contributor counts fluctuate freely and releases are community-driven, a perfectly invariant work rate is inherently unlikely. The law holds approximately but not strictly - hence partial applicability.

Law V - Conservation of Familiarity

"As an E-type system evolves, all associated with it must maintain mastery of its content and behaviour to achieve satisfactory evolution. Excessive growth diminishes that mastery. Thus the average incremental growth remains invariant."

Verdict: Applicable

Despite the overall codebase reaching ~5 million LOC and ~180,000 functions, the per-release increments in Number of Files hover around a mean of ~280 files, without growing proportionally to the total codebase size. This controlled per-release growth rate is consistent with the kernel developer community deliberately moderating the pace of change to maintain familiarity and mastery. The progression through major version series (v2.0 → v2.2 → v2.4 → v2.6) also shows gradual rather than abrupt growth in LOC and function counts per branch, suggesting conscious moderation of additions at each step.

Law VI - Continuing Growth

"The functional content of an E-type system must be continually increased to maintain user satisfaction over its lifetime."

Verdict: Strongly Applicable

Every single metric confirms continuous functional growth. LOC grew from ~0 to ~5 million. Functions grew from ~0 to over 180,000. System calls increased from ~130 to over 350. Across all major branches (v1.x, v2.0, v2.2, v2.4, v2.6), each new branch consistently begins at a higher baseline than its predecessor, confirming that substantial new functionality was added in each generation. This growth was necessary to support evolving hardware architectures, new network protocols,

additional file systems, multiprocessor support, and expanding user demands - a direct confirmation of Law VI.

Law VII - Declining Quality

"The quality of an E-type system will appear to decline unless it is rigorously maintained and adapted to operational environment changes."

Verdict: Partially Applicable

The Linux's Maintainability Index provides the most direct evidence here. Rather than showing a uniform decline, the index fluctuates and actually trends upward in later branches - particularly v2.4 and v2.6 - rising from ~108 toward ~115 by 2008. This suggests that the Linux kernel community actively worked to counteract quality decline through rigorous code reviews, subsystem refactoring, and strict release management. However, the McCabe Cyclomatic Complexity surpassing 700,000 means the system is objectively harder to reason about and maintain in absolute terms. The conclusion is that quality decline is being actively resisted and partially reversed - consistent with the second half of the law's statement ("unless rigorously maintained") - rather than being absent entirely.

Law VIII - Feedback System

"E-type evolution processes constitute multi-level, multi-loop, multi-agent feedback systems and must be treated as such to achieve significant improvement over any reasonable base."

Verdict: Applicable

The Linux kernel's development model is inherently feedback-driven at multiple levels. The Releases per Month chart shows surges of activity followed by calmer periods, consistent with feedback loops between testing, bug reporting, patching, and releasing. The Kernel Release Dates chart shows multiple major branches evolving concurrently (e.g., v2.0 and v2.2 overlapping), where stability issues discovered in the development branch feed back into the next stable release. The Linux kernel's well-known dual-track model - a stable branch and a development branch - is itself a formalised multi-loop feedback architecture. The controlled oscillation of per-release file increments around a mean further reflects macro-level feedback regulation of development pace.

Summary Table

Law	Name	Verdict
Law I	Continuing Change	Strongly Applicable
Law II	Increasing Complexity	Strongly Applicable
Law III	Self-Regulation	Partially Applicable
Law IV	Conservation of Organisational Stability	Partially Applicable
Law V	Conservation of Familiarity	Applicable
Law VI	Continuing Growth	Strongly Applicable
Law VII	Declining Quality	Partially Applicable
Law VIII	Feedback System	Applicable

Conclusion

The Linux kernel evolution data strongly supports the applicability of Lehman's Laws to large-scale open-source E-type systems. Laws I, II, VI, and VIII are most directly and clearly confirmed by the figures. Laws V and VIII are well-supported through the community's engineering discipline. Laws III, IV, and VII show partial applicability - not because the laws are wrong, but because the distributed, volunteer-driven nature of open-source development introduces variability not present in traditional commercial projects, and because the kernel community's active quality management practices actively counteract some of the predicted negative trends. Overall, the data validates Lehman's Laws as a robust framework for understanding long-term software evolution.

What is Software Evolution?

First, imagine you have a **mobile app** — say, a food delivery app. When it was first built, it was simple: just browse food and order. But over time:

- People wanted **live tracking**
- Then **multiple payment methods**
- Then **reviews and ratings**
- Then it needed to support **more restaurants, more cities**

So the developers kept **changing and adding things** to the app — year after year. This process of a software system **growing and changing over time** is called **Software Evolution**.

Now the question is: **is there a pattern to how software evolves?** Can we predict it? A scientist named **Manny Lehman** spent decades studying this.

Who is Manny Lehman?

Meir M. Lehman was a computer scientist who worked at IBM and later at Imperial College London. From the **1970s to the 1990s**, he studied how large real-world software systems change over time. He observed hundreds of software projects and noticed that software evolution is **not random** — it follows **predictable patterns**, which he called **Laws**.

Think of it like this: just as Newton observed falling objects and derived the **Laws of Motion**, Lehman observed evolving software and derived the **Laws of Software Evolution**.

What is an "E-type" System?

Before the laws, you need to understand that Lehman classified software into 3 types:

Type	Name	Meaning	Example
S-type	Specified	Solves a precisely defined mathematical problem. Output is always correct or wrong.	A program to calculate square roots
P-type	Problem-solving	Approximates solutions to real-world problems	A chess-playing program
E-type	Evolving	Embedded in and reflects the real world. Must change as the world changes.	Banking software, Linux, Facebook

Lehman's Laws apply specifically to E-type systems. The Linux kernel in your assignment is a perfect E-type system — it runs on real hardware, serves real users, and must keep evolving.

Lehman's 8 Laws — Explained Simply

Think of software like a **living city**. A city like Dhaka keeps growing, getting more complex, needing more maintenance. Lehman's laws describe exactly this kind of behaviour.

Law 1 — Continuing Change

"Software must keep changing or it becomes useless."

Simple analogy: Imagine WhatsApp stopped updating in 2015. No video calls, no stories, no reactions. People would leave and switch to something better.

In real life: If Linux stopped adding support for new processors, new security patches, new hardware — it would become irrelevant. So it **must** keep changing.

Law 2 — Increasing Complexity

"Every time you change software, it gets more complicated — unless you specifically work to simplify it."

Simple analogy: Imagine your school bag. First year: just books. Then you add a laptop, charger, water bottle, snacks, notebooks... it gets messier and heavier over time unless you occasionally clean it out.

In real life: Every new feature added to Linux adds new code, new connections between parts, new conditions to handle. It gets structurally more complex over time.

Law 3 — Self-Regulation

"The evolution of software is self-regulating. The amount of change per release stays roughly normal — not too much, not too little."

Simple analogy: Think of a classroom. Even without strict rules, students self-regulate — no one studies 20 hours one day then zero for a week. There's a natural average rhythm.

In real life: Linux releases tend to have a roughly consistent amount of change per version — not all over the place — suggesting a natural self-regulation in the development process.

Law 4 — Conservation of Organisational Stability

"The overall speed of development stays roughly the same throughout the life of the project."

Simple analogy: A bakery makes roughly the same number of loaves per week, year after year — even if individual bakers come and go, because the process and capacity regulate themselves.

In real life: Even though Linux has thousands of contributors and they change over years, the overall work output per period stays roughly stable.

Law 5 — Conservation of Familiarity

"You can't add too much new stuff at once, because developers need to stay familiar with the system."

Simple analogy: Imagine you're learning a new subject. You can absorb maybe 2–3 new concepts per day. If a teacher dumps 50 new concepts on you at once, you learn nothing. There's a natural limit.

In real life: Linux developers can only safely add so much per release — otherwise no one understands the new code well enough to maintain it properly.

Law 6 — Continuing Growth

"Software must keep adding features to keep users satisfied."

Simple analogy: Think of a restaurant. If the menu never changes and competitors are offering new dishes, customers leave. You must keep growing your offerings.

In real life: Linux must continuously add support for new hardware, new networking features, new security capabilities — or users and companies will switch to alternatives.

Law 7 — Declining Quality

"Software quality will appear to get worse over time unless active work is done to maintain it."

Simple analogy: Think of a road. Even a newly built road develops potholes and cracks over time due to traffic and weather. Unless the city actively repairs it, quality declines.

In real life: As Linux grows more complex, bugs accumulate, old code becomes outdated, and the system becomes harder to maintain — unless developers actively refactor, test, and clean up code.

Law 8 — Feedback System

"Software development is a feedback loop — decisions are based on what happened before, and results feed back into future decisions."

Simple analogy: Think of a thermostat. It checks the room temperature, compares it to the target, and adjusts the heater accordingly — constantly in a loop. Software development works the same way.

In real life: Linux developers look at bug reports from previous releases, user feedback, performance issues — and use all of that to decide what to do in the next release. It's a continuous loop.

Now — What is Your Assignment Asking?

Your assignment gives you **real charts from the Linux kernel project** (1994–2008) and asks: **do these charts actually show what Lehman's laws predict?**

Think of it like this — Lehman made **predictions** about how software behaves. Your job is to look at the **real data** and say: *"Yes, I can see this in the chart"* or *"This is not clearly visible"* and **explain why**.

Here is how each chart connects to the laws:

Chart from PDF	Which Law it Relates To
Number of System Calls (keeps rising)	Law 1, Law 6
LOC — Lines of Code (keeps rising)	Law 1, Law 6
Number of Functions (keeps rising)	Law 2, Law 6
McCabe Cyclomatic Complexity (keeps rising)	Law 2
Oman's Maintainability Index (stays roughly stable)	Law 7

Releases per Month (fluctuates but stays in a range)	Law 3, Law 4
Kernel Release Dates (steady release history)	Law 1, Law 8
Incremental Change in Files (oscillates around average)	Law 3, Law 4, Law 5

The Key Idea to Remember

Lehman's laws are saying: **large real-world software is like a living organism.** It must keep growing (or it dies), it naturally gets more complex, its development has a rhythm, and if you don't maintain it, it decays. The Linux kernel data is one of the best real-world examples that shows most of these laws in action.
